NLP-Based Plagiarism Detection and Text Similarity Analysis Using TF-IDF and Cosine Similarity

MOHAMED SHAH SHAJIR
24UBC249
III BCA B

GITHUB REPOSITORY LINK:

https://github.com/Mohamed-Shah-Shajir/NLP-Plagiarism-Detector

PROBLEM STATEMENT:

Plagiarism is the act of using or reproducing another person's work or ideas without proper acknowledgment. With the large amount of textual information available digitally, manually comparing documents to identify similar or copied content can be time-consuming and inefficient.

The objective of this project is to develop an NLP-based text similarity and plagiarism detection system that can automatically compare two text documents and determine how similar they are.

The system uses Natural Language Processing (NLP) techniques to clean and prepare text data. The cleaned text is converted into numerical representations using Term Frequency-Inverse Document Frequency (TF-IDF). The similarity between two texts is then calculated using Cosine Similarity.

Based on the similarity score, the system categorizes the text pair as Low Similarity, Moderate Similarity, or High Similarity. The system also allows users to enter two texts manually and obtain a plagiarism-related result.

OBJECTIVES:

The major objectives of this project are:

1. To develop an NLP-based system for detecting similarity between two text documents and determining whether the documents contain a significant amount of common textual information.
2. To collect, load, and analyze a suitable text similarity dataset containing pairs of documents that can be used for testing and evaluating the proposed similarity analysis approach.

3. To preprocess the textual data by converting text into a standard format and removing unnecessary elements such as punctuation, URLs, extra spaces, and common English stopwords.
4. To perform basic analysis of the dataset, including examining the number of records, missing values, duplicate records, text lengths, and other relevant characteristics of the textual data.
5. To convert the preprocessed text into numerical representations using TF-IDF vectorization, allowing the textual content to be processed mathematically by the similarity algorithm.
6. To calculate the similarity between text pairs using Cosine Similarity, which measures the similarity between the TF-IDF representations of the two documents.
7. To categorize the calculated similarity scores into Low Similarity, Moderate Similarity, and High Similarity categories based on predefined similarity thresholds.
8. To identify and analyze the most similar text pairs in the dataset, which can help demonstrate cases where documents contain highly similar or repeated content.
9. To identify the least similar text pairs and analyze the overall distribution of similarity scores, providing a better understanding of how textual similarity varies across the dataset.
10. To provide a simple user-input plagiarism checking feature using Python, where users can enter or provide two text documents and obtain a similarity score along with an interpretation of the similarity level.

DATASET

Dataset Description

The project uses a text similarity dataset containing pairs of textual documents. Each record contains two text fields that can be compared to determine their similarity.

The dataset is used to train the TF-IDF vocabulary and evaluate the similarity between corresponding text pairs.

The dataset contains news-style textual documents covering different topics such as sports, politics, technology, business, and social issues.

Dataset Source

Source: Kaggle

Dataset: Text Similarity Dataset

The dataset was downloaded from Kaggle and stored locally in the project directory.

The dataset file used in the project is:

Text_Similarity_Dataset.csv

Dataset Size

After removing incomplete text pairs, the dataset used for the similarity analysis contained:

Number of records: 4,023

Number of attributes: 3

Dataset Attributes

Attribute	Description
Unique_ID	Unique identifier for each text pair
text1	First text document
text2	Second text document

During the initial dataset inspection, one missing value was identified in the text2 column. Incomplete pairs were removed before performing the similarity analysis.

Classes / Categories

The original dataset does not provide plagiarism classes for the text pairs. Therefore, similarity categories were created based on the calculated cosine similarity score.

The categories used in the project are:

Similarity Score	**Category**
Score < 0.20	Low Similarity
0.20 ≤ Score < 0.50	Moderate Similarity
Score ≥ 0.50	High Similarity

These categories are rule-based categories created for this project, rather than original labels provided by the dataset.

Data Preparation

The following data preparation steps were performed:

1. The CSV dataset was loaded using Pandas.
2. The number of rows and columns was checked.
3. Column names and data types were examined.
4. Missing values were identified.
5. Incomplete text pairs were removed.
6. Duplicate records were checked.
7. Text length was analyzed using word counts.
8. Both text columns were prepared for NLP preprocessing.
9. The processed dataset was saved as:

processed_text_similarity_dataset.csv

METHODOLOGY:

The methodology followed in this project can be represented as:

Text Dataset → Data Cleaning → NLP Preprocessing → TF-IDF Vectorization → Cosine Similarity → Similarity Classification → Plagiarism Detection

The first stage involves loading the text similarity dataset using Python and Pandas. The dataset is examined to identify its number of rows, columns, missing values, duplicate records, and unique identifiers. Incomplete text pairs are removed before further processing.

In the next stage, NLP preprocessing is performed on both text columns. Each document is converted into lowercase to maintain consistency. URLs and punctuation are removed because they generally do not contribute significantly to the textual similarity calculation. The text is then divided into individual words, and common English stopwords are removed using the ENGLISH_STOP_WORDS collection provided by Scikit-learn. Finally, the remaining words are joined together to form the cleaned text.

After preprocessing, Term Frequency-Inverse Document Frequency (TF-IDF) is used to convert the text documents into numerical vectors. TF-IDF assigns importance to words based on their frequency in a document and their occurrence across the collection of documents. Words that occur frequently in a particular document but less frequently across the entire collection receive higher importance.

The next stage is calculating cosine similarity between the TF-IDF representation of text1 and text2 for every corresponding pair. Cosine similarity measures how similar two vectors are based on the angle between them. A value close to 1 indicates very high similarity, while a value close to 0 indicates very low similarity.

The resulting similarity score is then classified into three categories. A score of 0.50 or above is classified as High Similarity, a score from 0.20 to below 0.50 is classified as Moderate Similarity, and a score below 0.20 is classified as Low Similarity. For user-entered text, the system uses the similarity score to determine whether the pair should be considered potential plagiarism.

IMPLEMENTATION:

The project was implemented using the Python programming language. Development and execution were performed using Visual Studio Code, with a Python virtual environment created specifically for the project.

Several Python libraries were used during implementation. Pandas was used for loading, manipulating, analysing, and saving the dataset. NumPy was used for numerical operations. Matplotlib was used to visualize the distribution of similarity scores. The re and string modules were used for text cleaning and punctuation removal.

For NLP processing, Scikit-learn was used. The ENGLISH_STOP_WORDS collection was used to remove common English stopwords, while TfidfVectorizer was used to convert cleaned text into TF-IDF vectors. Cosine similarity was calculated between the corresponding TF-IDF vectors to obtain a numerical similarity score.

The implementation first loads the dataset and performs basic analysis. After preprocessing, both text columns are combined to create a common vocabulary for TF-IDF. The vectorizer is fitted on the combined documents and then used to transform text1 and text2 separately. The corresponding vectors are compared using cosine similarity. The generated scores are stored in the dataset along with their similarity categories.

The project also includes a user-input plagiarism checker. The user can enter two pieces of text through the terminal. The system preprocesses both inputs, creates their TF-IDF representations, calculates their cosine similarity, and displays the similarity score, similarity percentage, similarity category, and final result.

RESULTS:

The implemented system successfully processed the selected text similarity dataset and calculated similarity scores for all 4,023 complete text pairs. During TF-IDF processing, 8,046 documents were used to build a common vocabulary. The resulting TF-IDF matrices had the shape (4023, 32440) for both text1 and text2, meaning that 32,440 vocabulary terms were identified from the combined text collection.

The statistical analysis of the similarity scores produced a mean similarity of approximately 0.0289, with a minimum score of 0.0000 and a maximum score of 1.0000. The median similarity score was approximately 0.0176. These results indicate that most of the text pairs in the dataset have relatively low similarity, while a small number of pairs contain highly similar content.

The similarity category distribution showed that 3,990 pairs were classified as Low Similarity, 29 pairs as Moderate Similarity, and 4 pairs as High Similarity.

Similarity Category	**Number of Pairs**
Low Similarity	3,990
Moderate Similarity	29
High Similarity	4
Total	**4,023**

The system also successfully identified the most similar text pairs. One pair, with Unique ID 3403, obtained a similarity score of 1.0000, indicating that the two texts were effectively identical after preprocessing. Another highly similar pair, with Unique ID 2284, obtained a score of approximately 0.9592.

The user-input testing also demonstrated that the system can detect highly similar short texts. For example, when the inputs “hello world” and “hello guyz”

were provided, the system produced a similarity score of approximately 0.963, corresponding to a similarity percentage of 96.3% and a High Similarity classification. The system therefore returned “Potential Plagiarism” for this test case.

A histogram of the cosine similarity scores can be included in this section to visually demonstrate the distribution of similarity values across the dataset. Screenshots of the terminal output showing the TF-IDF results, similarity statistics, Top 10 Similar Text Pairs, and user-input plagiarism detection can also be included as supporting evidence.

LIMITATIONS:

Although the developed system successfully performs text similarity analysis, it has several limitations. The system primarily relies on TF-IDF, which focuses on the occurrence and importance of individual words. Therefore, it may not fully understand the semantic meaning or context of a document. Two texts that express the same idea using completely different words may receive a relatively low similarity score.

The system may also produce high similarity scores when two texts contain many common words even if the actual meaning or context is different. The effectiveness of the system is also influenced by the vocabulary present in the dataset. Since the current implementation is mainly based on English text and English stopwords, it does not provide proper multilingual plagiarism detection.

Another limitation is that the system uses predefined similarity thresholds to categorize results as low, moderate, or high similarity. These thresholds are manually selected and may not be appropriate for every type of document or application. The current implementation is also a standalone Python-based system rather than a web or mobile application.

FUTURE SCOPE:

The project can be improved in several ways in the future. A larger and more diverse dataset can be used to improve the reliability of the similarity analysis. More advanced NLP preprocessing techniques such as stemming, lemmatization, and named entity handling can also be incorporated.

The TF-IDF approach can be extended or replaced with advanced semantic representation techniques such as Word2Vec, GloVe, BERT, Sentence-BERT, or other transformer-based models. These approaches can provide better semantic understanding and can detect similarities even when different words are used to express the same meaning.

The system can also be extended to support multiple languages, making it useful for multilingual plagiarism detection. Another possible improvement is to introduce a more comprehensive evaluation framework using labelled plagiarism datasets and metrics such as precision, recall, F1-score, and accuracy.

In addition, the current standalone application can eventually be converted into a web or mobile application where users can upload documents or paste text and receive an automated plagiarism report. Future versions could also highlight the specific sentences or phrases that are similar between two documents, making the system more useful for academic and professional applications.

The system can also be improved by introducing adaptive similarity thresholds instead of using fixed threshold values. This would allow the system to determine similarity levels more effectively for different types of documents and domains. Additional techniques could also be explored to reduce false positives and improve the accuracy of plagiarism detection.

Another useful enhancement would be to maintain a database of previously submitted documents and compare new documents against the stored collection. This could help identify similarities across a large number of documents and make the system more suitable for educational institutions, where assignments and academic submissions need to be checked regularly.

CONCLUSION:

This project successfully developed an NLP-based text similarity and plagiarism detection system using TF-IDF and cosine similarity. The system processes raw text, performs NLP preprocessing, converts the cleaned documents into numerical TF-IDF representations, and calculates the similarity between corresponding text pairs using cosine similarity.

The analysis was performed on 4,023 complete text pairs, with 8,046 documents contributing to the TF-IDF vocabulary. The system identified 32,440 vocabulary terms and successfully calculated similarity scores for the complete dataset. The results showed that most text pairs had low similarity, while a small number of pairs demonstrated moderate or high similarity. The system also successfully identified highly similar document pairs, including a pair with a similarity score of 1.0.

A terminal-based user plagiarism checker was additionally implemented, allowing two texts to be entered and compared directly. The system produces a similarity score, percentage, similarity category, and plagiarism indication. Through this project, the practical application of NLP preprocessing, TF-IDF vectorization, cosine similarity, data analysis, and text similarity detection was demonstrated. The project also provided an understanding of the limitations of traditional word-based similarity methods and the potential for improving plagiarism detection using advanced NLP and semantic models.